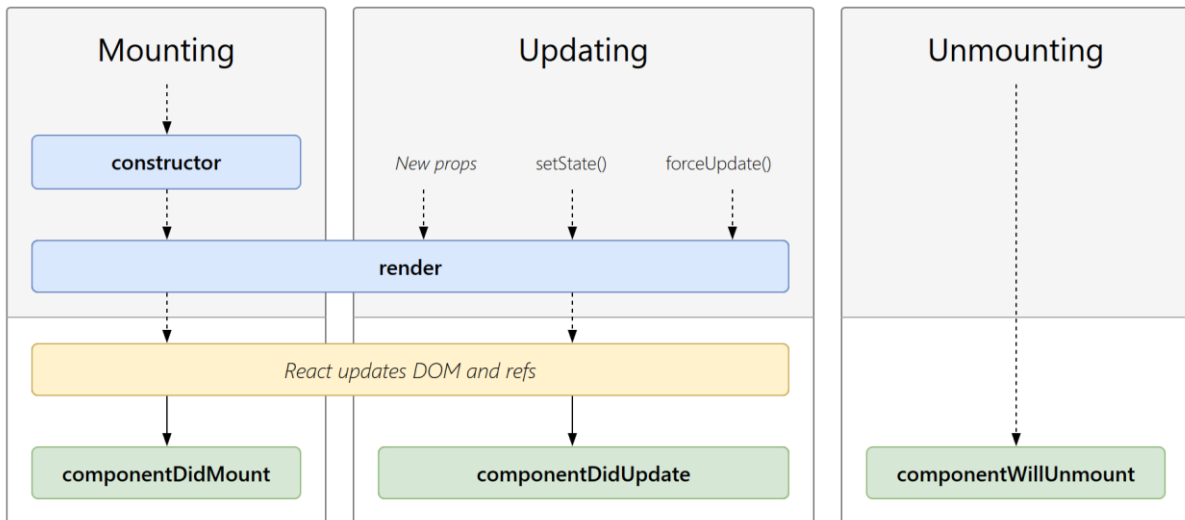


useEffect()



`useEffect()` en React

`useEffect` est un hook qui permet de gérer les **effets de bord** (side effects) dans les composants fonctionnels. Il remplace les méthodes `componentDidMount`, `componentDidUpdate`, et `componentWillUnmount` des composants classe.

Syntaxe de base :

```
useEffect(() => {
  // Code à exécuter lors du montage du composant
  console.log('Composant monté');

  // Code à exécuter lors du démontage du composant
  return () => {
    console.log('Composant démonté');
  };
}, []); // Le tableau vide signifie que cet effet s'exécute uniquement
lors du montage et du démontage
```

Cas d'utilisation courants :

1. Exécuter une fois après le premier rendu (équivalent à `componentDidMount`)

```
useEffect(() => {
  console.log("Exécuté une fois après le montage du composant");
}, []); // Tableau de dépendances vide
```

2. Exécuter à chaque rendu (sans tableau de dépendances)

```
useEffect(() => {
  console.log("Exécuté à chaque rendu");
});
```

3. Exécuter quand une variable change (équivalent à `componentDidUpdate`)

```
const [count, setCount] = useState(0);
useEffect(() => {
  console.log("Count a changé :", count);
}, [count]); // Déclenché quand `count` est modifié
```

4. Nettoyer les effets (équivalent à `componentWillUnmount`)

```
useEffect(() => {
  // Code à exécuter (effet de bord)
  return () => {
    // Nettoyage (optionnel)
  };
}, []);
```

`useEffect` permet de dire à React d'exécuter une fonction **après** que le composant ait été rendu à l'écran (ou mis à jour). Cette fonction est l'effet secondaire.

On va revenir à l'exemple des hooks on décommente la ligne de hooks et on supprimer les props.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import Hooks from './components/Hooks';
import Form from './components/Form';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  //<Form />
  <Hooks />
  // <App />
);
```

On crée un autre bouton pour décrémenter le nombre qu'on a.

```
import React, { useState } from 'react';
import './App.css';
```

```
function Hooks (props) {
  const [count,setCount]=useState(0);
  function hundelClickAjout(){
    setCount(count+1)
  }
  function hundelClickRduit(){
    setCount(count-1)
  }
  return (
    <div>
      vous avez cliquez {count}
      <br/>
      <button onClick={hundelClickAjout}> Incrementer par 1 </button>
      <button onClick={hundelClickRduit}> Decrementer par 1 </button>
    </div>
  );
}
export default Hooks;
```

L'exercice qu'on va faire c'est que si le nombre devient négatif on va le colorier en rouge.

On va ajouter la fonction useEffect avec un cosole.log pour voir qu'elle s'exécute.

```
import React, { useEffect, useState } from 'react';
import '../App.css';

function Hooks (props) {
  const [count,setCount]=useState(0);
  function hundelClickAjout(){
    setCount(count+1)
  }
  function hundelClickRduit(){
    setCount(count-1)
  }

  useEffect(function(){
    console.log(count + "a changé")
  },[count])

  return (
    <div>
      vous avez cliquez {count}
      <br/>
      <button onClick={hundelClickAjout}> Incrementer par 1 </button>
      <button onClick={hundelClickRduit}> Decrementer par 1 </button>
    </div>
  );
}
```

```
export default Hooks;
```

Après on modifie le style du div qui contient l'affichage du message.

```
import React, { useEffect, useState } from 'react';
import '../App.css';

function Hooks (props) {
  const [count, setCount]=useState(0);
  function hundeClickAjout(){
    setCount(count+1)
  }
  function hundeClickRedit(){
    setCount(count-1)
  }
}

useEffect(function(){
  // console.log(count + "a changé")
  if (count<0)
    document.getElementById("divMessage").style.color="red"
  else
    document.getElementById("divMessage").style.color="black"

},[count])

  return (
    <div id="divMessage">
      vous avez cliquez {count}
      <br/>
      <button onClick={hundeClickAjout}> Incrementer par 1 </button>
      <button onClick={hundeClickRedit}> Decrementer par 1 </button>
    </div>
  );
}
export default Hooks;
```

Comment faire pour que ça ne s'exécute QUE si compteur change (et PAS au premier rendu) ?

La solution la plus courante est d'utiliser une ref pour savoir si c'est le premier rendu ou non.

```
function Hooks(props) {
  const [compteur, SetCompteur]=useState(0)
  const premiereFois=useRef(true)
```

```

    function handelClickI(){
        SetCompteur(compteur+1)
    }
    function handelClickD(){
        SetCompteur(compteur-1)
    }

    useEffect(function(){
        if (premiereFois.current)
            premiereFois.current=false
        else
            console.log("je suis executé")
    },[compteur])

```

- **useRef(true)** : On crée une ref (premiereFois) initialisée à true. Les refs persistent entre les rendus sans déclencher de nouveau rendu quand elles changent.

- **Premier Rendu :**

- useEffect s'exécute car c'est le premier rendu.
- La condition if (premiereFois.current) est vraie.
- On met premiereFois.current à false.
- Le else n'est pas exécuté. Rien n'est loggué (sauf si vous ajoutez un log dans le if).

Faire des itérations et créer des listes

Revenons à l'exemple des composants du prerequis. Imaginons qu'au lieu de HTML, CSS ET JAVASCRIPT, on a des dizaines de technologies, alors la méthode que nous avons utilisée n'est pas performante.

Nous allons créer un tableau `technologies` qui contient les outils sous forme d'objets avec (id, nom, image).

On va utiliser le Mapping dans le render de Prerequis.js : Utilisation de la méthode `.map()` pour parcourir le tableau des technologies et rendre un `ElementPrerequis` pour chaque technologie, en lui passant les informations via les props.

On va décommenter `<App />` dans `index.js` et commenter hook

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import Hooks from './components/Hooks';
import Form from './components/Form';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(

```

```
//<Form />
//<Hooks />
<App />

);
```

Puis on va changer notre composant `prerequis.js`

```
import React from 'react';
import '../App.css';
import ElementPrerequis from './ElementPrerequis';

function Prerequis () {
  const technologies = [
    {id: "html", nom: "HTML", image: "\\images\\html.png"},
    {id: "css", nom: "CSS", image: "\\images\\css.png"},
    {id: "javascript", nom: "JavaScript", image: "\\images\\javascript.png"}
  ]
  return (
    <ul>
      {
        technologies.map((technologie) => (
          <ElementPrerequis key={technologie.id} id={technologie.id}
nom={technologie.nom} image={technologie.image}/>
        ))
      }
    </ul>
  );
}
export default Prerequis;
```

Mapping dans le `render` de `Prerequis.js` : Utilisation de la méthode `.map()` pour parcourir le tableau des technologies et rendre un `ElementPrerequis` pour chaque technologie, en lui passant les informations via les props.

Et si on utilise

```
technologies.forEach(technologie => {
  <ElementPrerequis key={technologie.id} id={technologie.id}
nom={technologie.nom} image={technologie.image}/>
})
```

Au lieu de :

```
technologies.map(technologie => (
  <ElementPrerequis key={technologie.id} id={technologie.id}
nom={technologie.nom} image={technologie.image}/>
))
```

)

Ça ne fonctionnera pas car :

forEach est conçu pour *exécuter* une fonction pour chaque élément d'un tableau, mais il ne *retourne* pas un nouveau tableau. En React, vous devez retourner un tableau de composants à rendre. map, en revanche, itère sur le tableau et retourne un nouveau tableau contenant les résultats de la fonction appliquée à chaque élément. C'est exactement ce dont nous avons besoin pour générer la liste des ElementPrerequis. On peut faire un test et essayer le code suivant pour comprendre :

```
import React from 'react';
import ElementPrerequis from './ElementPrerequis';
function Prerequis () {
  const technologies=[
    { id : "html", nom: "HTML", image: "/images/html.png"},
    { id : "css", nom: "CSS", image: "/images/css.png"},
    { id : "javascript", nom: "JavaScript", image: "/images/javascript.png"}
  ]
  const liste=technologies.map(technologie=>(
    <ElementPrerequis key={technologie.id} id={technologie.id}
    nom={technologie.nom} image={technologie.image}/>
  ))
  return (
    <ul>
      {
        liste
      }
    </ul>
  );
}
```

Par contre si on fait la même chose pour forEach ça ne fonctionnera pas.

Même sur le console, les éléments par map s'affichent et les éléments forEach ne s'affichent pas.

Pourquoi ajoute-t-on key dans la méthode .map () ?

Problème sans key

Lorsqu'on utilise map () pour afficher une liste d'éléments React, chaque élément doit avoir une **clé unique (key)**.

Si on ne met pas de key, React affiche un **avertissement dans la console** et peut rencontrer des problèmes de performance et d'affichage.

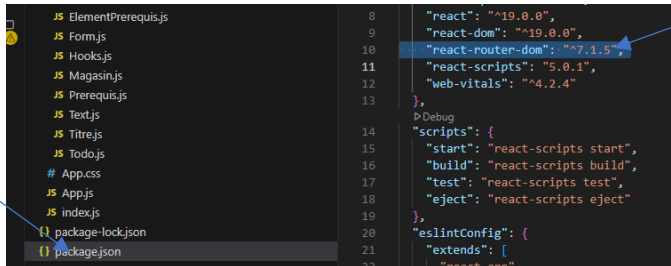
Chaque élément doit être identifiable de manière **unique** pour que React sache **quel élément mettre à jour, ajouter ou supprimer**.

Le routage en React

Étapes simples pour utiliser React Router :

Installer React Router

npm install react-router-dom



On va sur index.js et on décommente <app/>

On va sur app.js et ajouter notre browser pour les routes

```
import React from 'react';
import './App.css';
import Todo from './components/Todo';
import { BrowserRouter, Route, Routes } from 'react-router-dom';
import Magasin from './components/Magasin';
import Form from './components/Form';

function App () {
  return (
    // <div >
    //   <Todo/>
    // </div>
    <BrowserRouter>
    <Routes>
      <Route path="/" element={<Todo/>} />
      <Route path="/produits" element={<Magasin/>} />
      <Route path="/contact" element={<Form/>} />
    </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Maintenant on peut tester sur le navigateur :

<http://localhost:3000/produits>

<http://localhost:3000/contact>

et <http://localhost:3000/>

On crée un nouveau composant Navbar.js

```
import React from 'react';
import { Link } from 'react-router-dom';
function Navbar () {
  return (
    <nav className="navbar navbar-light bg-light">
      <form className="container-fluid justify-content-start">
        <Link to= "/" className="btn btn-outline-success me-2" >Accueil</Link>
        <Link to= "/Produits" className="btn btn-outline-success me-2"
      >Accueil</Link>
        <Link to= "/Contact" className="btn btn-outline-success me-2"
      >Accueil</Link>
      </form>
    </nav>

  );
}
export default Navbar;
```

Ensuite on appelle ce composant depuis notre browserRouter

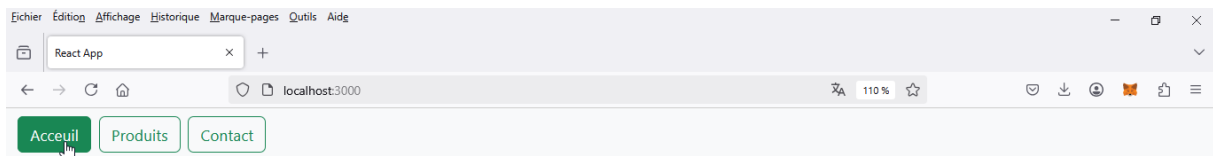
```
import React from 'react';
import './App.css';
import Todo from './components/Todo';
import { BrowserRouter, Route, Routes } from 'react-router-dom';
import Magasin from './components/Magasin';
import Form from './components/Form';
import Navbar from './components/Navbar';

function App () {
  return (
    // <div >
    //   <Todo/>
    // </div>
    <BrowserRouter>
    <Navbar />
    <Routes>
      <Route path="/" element={<Todo/>} />
      <Route path="/produits" element={<Magasin/>} />
      <Route path="/contact" element={<Form/>} />
    </Routes>
  );
}
```

```

    </Routes>
  </BrowserRouter>
);
}
export default App;

```



Introduction à React

React est une bibliothèque JavaScript populaire utilisée pour construire des interfaces utilisateur (UI). Développée par Facebook, elle permet de créer des applications web interactives et dynamiques de manière efficace. Avant de commencer avec React, il est utile de maîtriser certains concepts et technologies de base. Voici une liste des éléments clés à connaître :

-  HTML
-  CSS
-  JavaScript

Page 404

On crée une page 404 qui sera afficher au cas où on indique une route qui n'existe pas dans notre application.

On commence par créer un nouveau composant NotFound.js

```

import React from 'react';

function NotFound() {
  return (
    <div>
      <h1>Erreur 404</h1>
      <p>Désolé, la page que vous recherchez n'existe pas.</p>
      <a href="/">Retourner à l'accueil</a>
    </div>
  );
}

export default NotFound;

```

Puis on va l'ajouter dans notre BrowserRouter

```

import React from 'react';
import './App.css';
import Todo from './components/Todo';

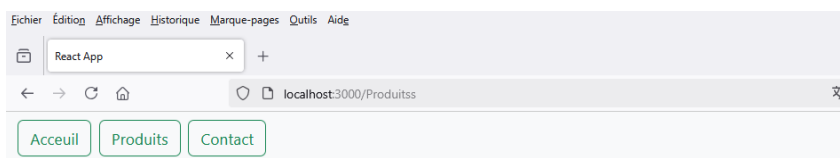
```

```
import { BrowserRouter, Route, Routes } from 'react-router-dom';
import Magasin from './components/Magasin';
import Form from './components/Form';
import Navbar from './components/Navbar';
import NotFound from './components/NotFound';

function App (){

  return (
    // <div >
    //   <Todo/>
    // </div>
    <BrowserRouter>
    <Navbar />
    <Routes>
      <Route path="/" element={<Todo/>}/>
      <Route path="/produits" element={<Magasin/>}/>
      <Route path="/contact" element={<Form/>}/>
      <Route path="*" element={<NotFound/>}/>

    </Routes>
    </BrowserRouter>
  );
}
export default App;
```



Erreur 404

Désolé, la page que vous recherchez n'existe pas.

[Retourner à l'accueil](#)